CLOUD COMPUTING LABORATORY-2

GROUP NUMBER	GROUP MEMBER	MAIL_ID
20	RUSHIL LINGAIAH CHERUKUPALLY	RUCH24@STUDENT.BTH.SE

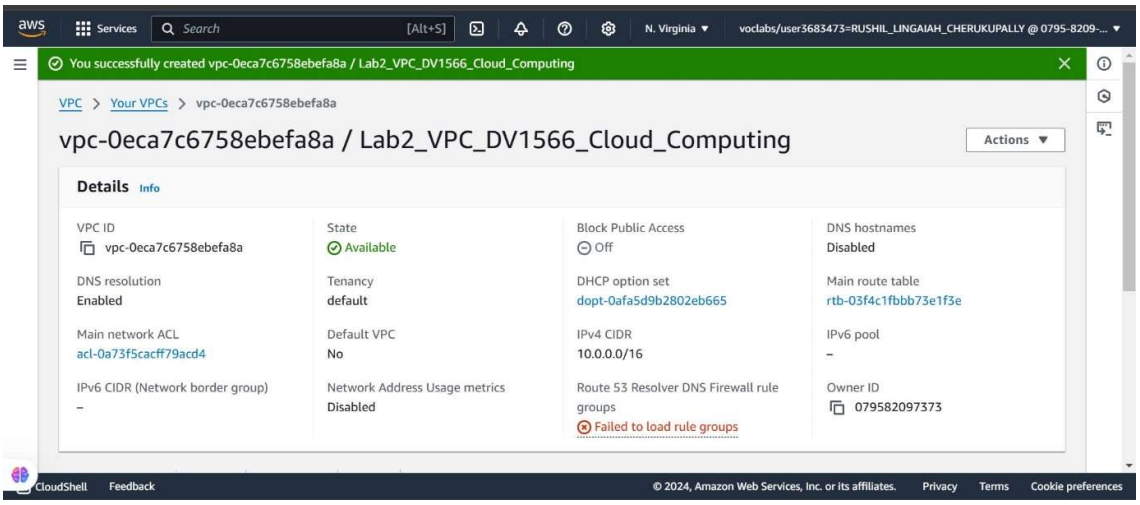
TASK 1: Setting Up and Running an Amazon EC2 Instance

1. Initial Setup:

I started by logging into the AWS Learner Lab using the provided Single Sign-On (**SSO**) URL. This ensured secure and role-based access to the AWS Management Console—a critical step for maintaining the integrity of the environment.

2. Creating a VPC:

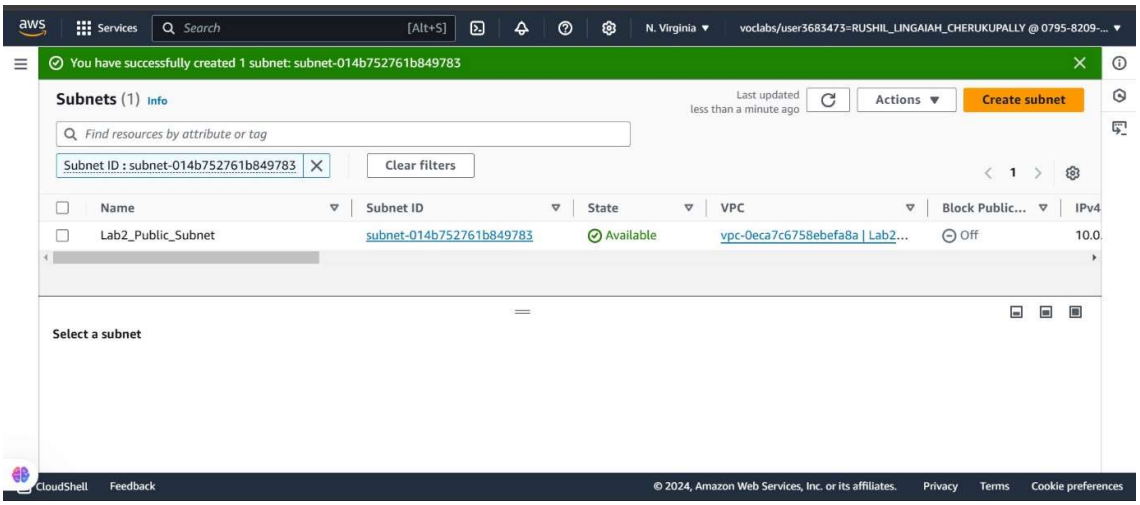
To establish an isolated network, I have created a Virtual Private Cloud (VPC) named **Lab2_VPC_DV1566_Cloud_Computing**. I assigned it an IPv4 CIDR block of 10.0.0.0/16, providing a private address space for deploying resources. I kept the default settings to simplify the setup while preparing for EC2 instance deployment.

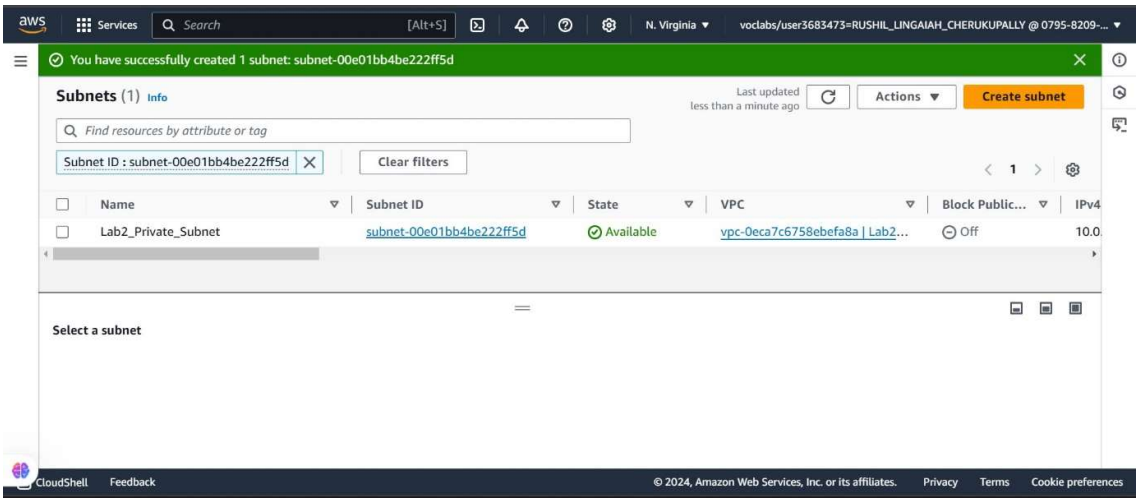


As part of the VPC setup, two subnets were created:

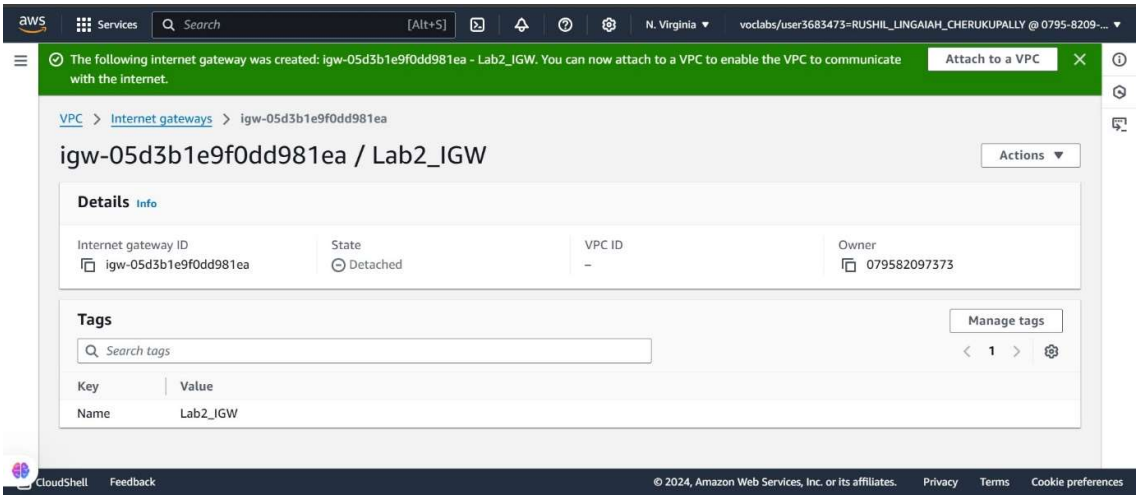
- **Public Subnet:** Named **Lab2_Public_Subnet**, with the CIDR block **10.0.1.0/24**, this subnet allows external access to resources such as web servers by routing traffic through an Internet Gateway.
- **Private Subnet:** Named **Lab2_Private_Subnet**, with the CIDR block **10.0.2.0/24**, this subnet is isolated from internet access and intended for hosting internal resources like databases, ensuring secure and restricted connectivity.

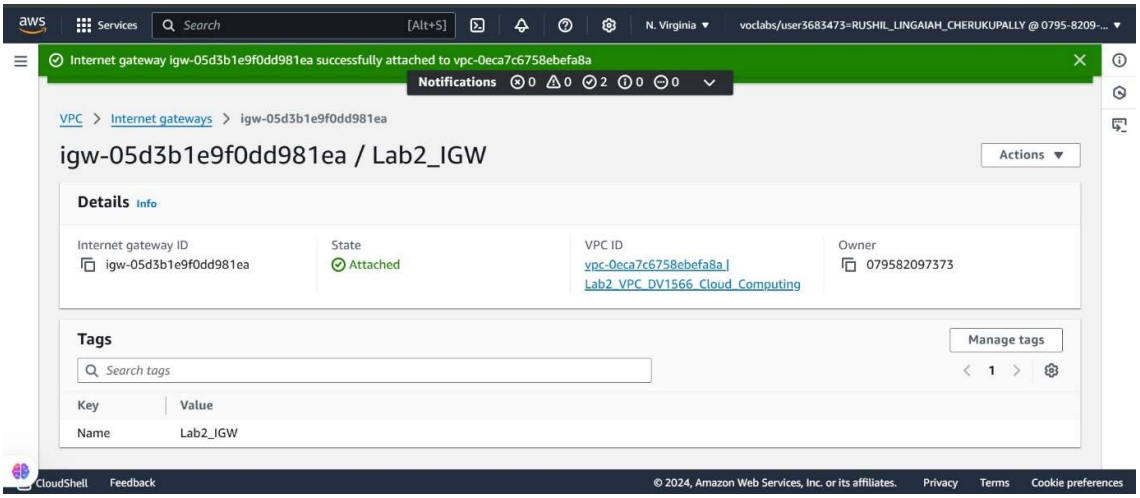
This separation ensures a secure and efficient network design tailored to the needs of the deployed resources.





And for VPC to communicate with the internet an Internet Gateway with the name **Lab2_IGW** is created and attached to the **Lab2_VPC_DV1566_Cloud_Computing**



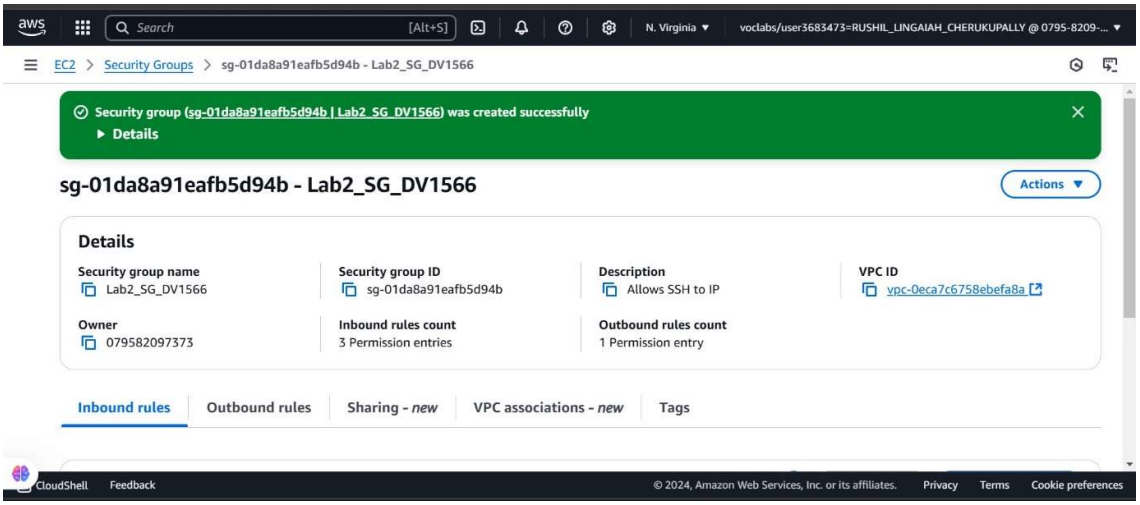


3. Configuring Security Groups:

The next step was to create a security group, which I named **Lab2_SG_DV1566** to manage traffic to and from the EC2 instance. Key inbound rules included:

- **SSH (Port 22):** Allowed secure shell access for managing the instance.
- **HTTP (Port 80):** Enabled web traffic for HTTP communication.
- **HTTPS (Port 443):** Allowed secure web traffic with encrypted connections.

The default outbound rule permitted all traffic, ensuring the instance could access updates and external services.

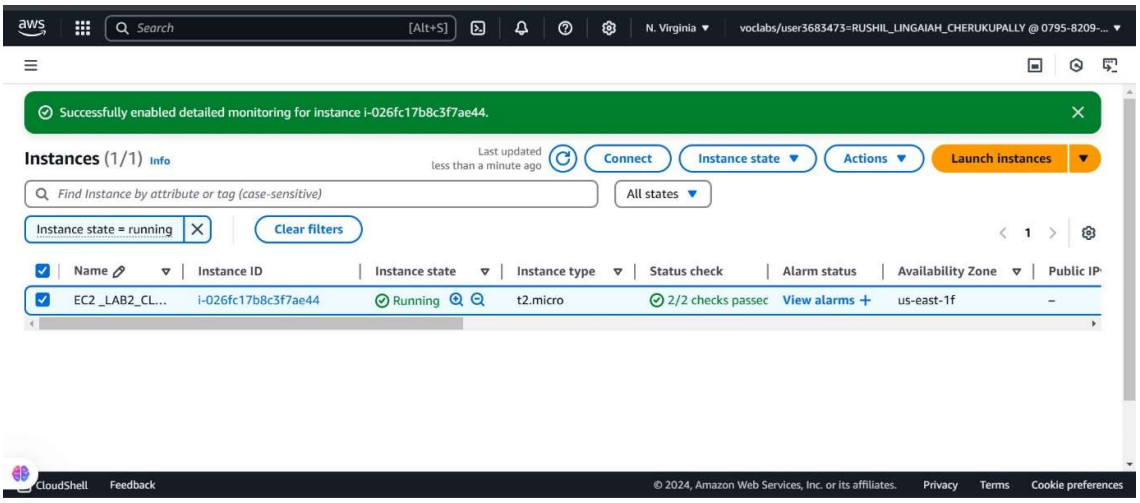


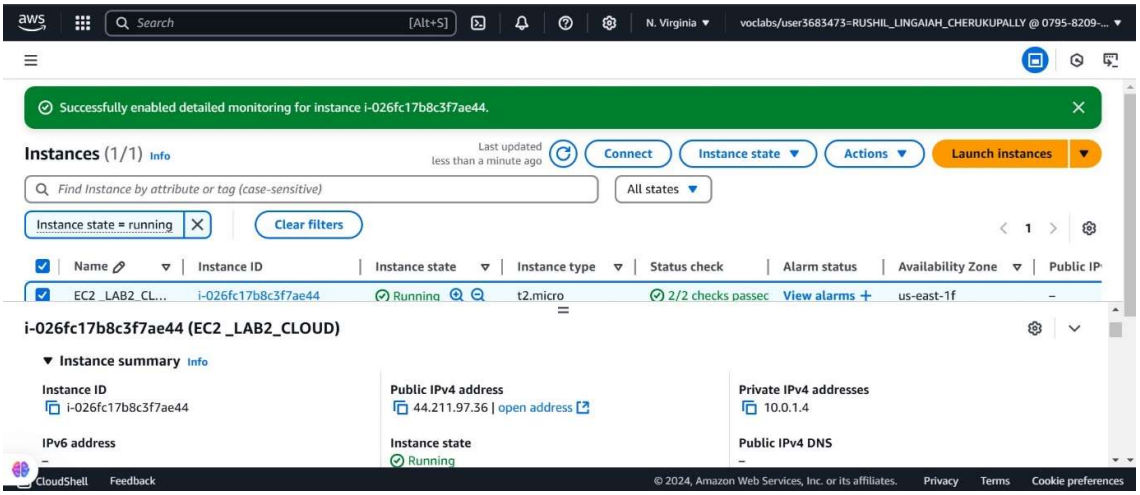
4. Routing Internet Traffic:

To enable internet access for the EC2 instances, I updated the route table to include a default route pointing to an internet gateway. Associating the public subnet with this updated route table ensured proper traffic routing.

5. Launching the EC2 Instance:

I deployed an EC2 instance using the **Amazon Linux 2 AMI** for its performance and stability. Named **EC2_LAB2_CLOUD**, the instance was launched within the VPC and associated with the configured security group and subnet. For secure access, I created an RSA key pair named **Lab2_Key**.





6. Connecting to the Instance:

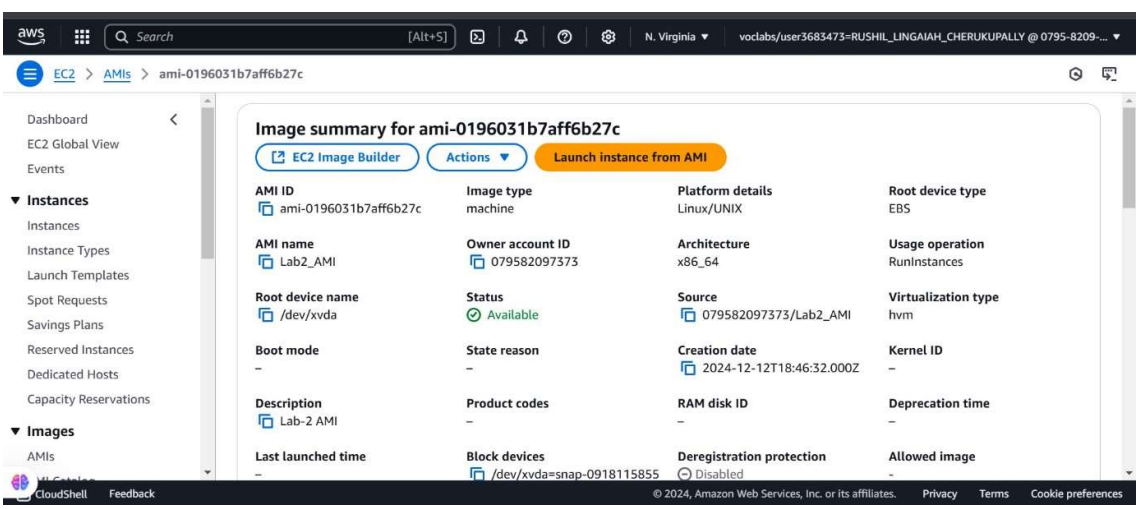
Once launched, I connected to the EC2 instance using PuTTY. Providing the instance's public IPv4 address and the private key for authentication, I verified the instance's functionality with basic commands:

sudo yum update -y

sudo amazon-linux-extras enable epel

7. AMI Creation:

Navigating to the **AMIs** section of the AWS Management Console to view the details of the custom Amazon Machine Image (AMI) created for deployment, with the name **Lab2_AMI.**



TASK 2: Monitoring Virtual Machines

1. Updating the System:

To secure the environment, I obtained root privileges (sudo su) and updated all packages using:

sudo yum update -y

```
Using username "ec2-user".
Authenticating with public key "LAB2_KEY_CLOUD"
Last login: Fri Dec 13 17:38:16 2024 from host-193-11-184-5.eduroam.bth.se
       __|  __|_  )
       _|  (     /   Amazon Linux 2
      ___|\___|___|
                      AL2 End of Life is 2025-06-30.

A newer version of Amazon Linux is available!

                      Amazon Linux 2023, GA and supported until 2028-03-15.
                        https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-1-4 ~]$ sudo su
[root@ip-10-0-1-4 ec2-user]# sudo yum install sysbench -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core                                               | 3.6 kB     00:00
230 packages excluded due to repository priority protections
Package sysbench-1.0.17-2.el7.x86_64 already installed and latest version
Nothing to do
[root@ip-10-0-1-4 ec2-user]# sudo sysbench --test=cpu --cpu-max-prime=200000
run
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
    events per second:    37.74

General statistics:
    total time:                          10.0128s
    total number of events:              378
```

2. Installing sysbench:

To benchmark system performance, I installed **sysbench** with:

sudo yum install sysbench -y

Then I conducted a CPU test with:

sudo sysbench --test=cpu --cpu-max-prime=200000 run

This provided metrics such as execution time and throughput.

```
[root@ip-10-0-1-4 ec2-user]# sudo sysbench --test=cpu --cpu-max-prime=200000
run
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
    events per second:    37.74

General statistics:
    total time:                          10.0128s
    total number of events:              378

Latency (ms):
         min:                                    24.95
         avg:                                    26.48
         max:                                    42.20
         95th percentile:                        27.17
         sum:                                 10009.76

Threads fairness:
    events (avg/stddev):           378.0000/0.00
    execution time (avg/stddev):   10.0098/0.00
```

3. Performing CPU Stress Tests:

Using a script, I ran CPU stress tests across multiple thread configurations:

> **for each in 1 2 3 8 16 32 64; do sysbench --test=cpu --cpu-max-prime=200000 --num-threads=$each run; done**

This test measured scalability and performance under different workloads.

```
[root@ip-10-0-1-4 ec2-user]# for each in 1 2 4 8 16 32 64; do
> sysbench --test=cpu --cpu-max-prime=200000 --num-threads=$each run;
> done
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
    events per second:    37.75

General statistics:
    total time:                          10.0106s
    total number of events:              378

Latency (ms):
         min:                                    25.19
         avg:                                    26.47
         max:                                    30.15
         95th percentile:                        27.17
         sum:                                  10006.60

Threads fairness:
    events (avg/stddev):           378.0000/0.00
    execution time (avg/stddev):   10.0066/0.00

WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
WARNING: --num-threads is deprecated, use --threads instead
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 2
Initializing random number generator from current time
```

```
Running the test with following options:
Number of threads: 32
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
    events per second:    37.75

General statistics:
    total time:                          10.2513s
    total number of events:              387

Latency (ms):
         min:                                   263.63
         avg:                                   832.51
         max:                                  1065.67
         95th percentile:                       943.16
         sum:                                 322180.31

Threads fairness:
    events (avg/stddev):           12.0938/0.29
    execution time (avg/stddev):   10.0681/0.08

WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
WARNING: --num-threads is deprecated, use --threads instead
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 64
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!
```

```
Running the test with following options:
Number of threads: 64
Initializing random number generator from current time


Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
    events per second:    37.76

General statistics:
    total time:                          10.6700s
    total number of events:              403

Latency (ms):
         min:                                    637.61
         avg:                                   1621.62
         max:                                   2137.34
         95th percentile:                       1869.60
         sum:                                 653513.06

Threads fairness:
    events (avg/stddev):           6.2969/0.46
    execution time (avg/stddev):   10.2111/0.25
```

4. Conducting Disk Stress Tests:

To assess disk performance, I prepared disk files with:

sysbench --test=fileio --file-total-size=8G --file-num=3 prepare

There was an error while showing there is no space left for writing the file. As shown in the screenshot below:

```
[root@ip-10-0-1-4 ec2-user]# sysbench --test=fileio --file-total-size=8G --fi
le-num=3 prepare
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

3 files, 2796202Kb each, 8191Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
Creating file test_file.1
Creating file test_file.2
FATAL: Failed to write file! errno = 28 (No space left on device)
```

Then to solve the error, I changed the file size from 8G to 1G and with the command as shown below, I performed the disk processing.

sysbench --test=fileio --file-total-size=1G --file-num=3 prepare

I then ran stress tests with varying thread counts to evaluate read/write speeds and latency:

for each in 1 4 8 16 32; do sysbench --test=fileio --file-total-size=1G --file-test-mode=rndwr --max-time=240 --max-requests=0 --file-block-size=4K --file-num=3 --num-threads=$each run; done

```
[root@ip-10-0-1-4 ec2-user]# sysbench --test=fileio --file-total-size=1G --fi
le-num=3 prepare
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

3 files, 349525Kb each, 1023Mb total
Creating files for the test...
Extra file open flags: (none)
Reusing existing file test_file.0
Reusing existing file test_file.1
Reusing existing file test_file.2
No bytes written.
```

```
Latency (ms):
         min:                                    0.00
         avg:                                    0.53
         max:                                  130.66
         95th percentile:                        2.14
         sum:                               239191.81

Threads fairness:
    events (avg/stddev):           453978.0000/0.00
    execution time (avg/stddev):   239.1918/0.00

sysbench 1.0.17 (using system LuaJIT 2.0.4)

Running the test with following options:
Number of threads: 4
Initializing random number generator from current time


Extra file open flags: (none)
3 files, 341.33MiB each
1024MiB total file size
Block size 4KiB
Number of IO requests: 0
Read/Write ratio for combined random IO test: 1.50
Periodic FSYNC enabled, calling fsync() each 100 requests.
Calling fsync() at the end of test, Enabled.
Using synchronous I/O mode
Doing random write test
Initializing worker threads...

Threads started!
```

5. CLEANING UP THE FILES:

After completing the above steps successfully, I have cleaned up the storage with the command:

sysbench –test-fileio –file-total-size=1G –file-num-3 cleanup

```
Threads started!

sysbench --test=fileio --file-total-size=1G --file-num=3 cleanup


File operations:
    reads/s:                      0.00
    writes/s:                     2172.79
    fsyncs/s:                     65.58

Throughput:
    read, MiB/s:                  0.00
    written, MiB/s:               8.49

General statistics:
    total time:                          240.0267s
    total number of events:              537175

Latency (ms):
         min:                                    0.00
         avg:                                   14.30
         max:                                  201.56
         95th percentile:                       61.08
         sum:                             7679268.11

Threads fairness:
    events (avg/stddev):           16786.7188/191.26
    execution time (avg/stddev):   239.9771/0.01

[root@ip-10-0-1-4 ec2-user]#
[root@ip-10-0-1-4 ec2-user]# sysbench --test=fileio --file-total-size=1G --filesysbench --test=fileio --file-total-size=1G --file-num=3 cleanup
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)
```

```
Latency (ms):
         min:                                    0.00
         avg:                                   14.30
         max:                                  201.56
         95th percentile:                       61.08
         sum:                              7679268.11

Threads fairness:
    events (avg/stddev):           16786.7188/191.26
    execution time (avg/stddev):   239.9771/0.01

[root@ip-10-0-1-4 ec2-user]#
[root@ip-10-0-1-4 ec2-user]# sysbench --test=fileio --file-total-size=1G --filesysbench --test=fileio --file-total-size=1G --file-num=3 cleanup
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

invalid option: --filesysbench
[root@ip-10-0-1-4 ec2-user]#
[root@ip-10-0-1-4 ec2-user]# sysbench --test=fileio --file-total-size=1G --fi
le-num=3 cleanup
WARNING: the --test option is deprecated. You can pass a script name or path
on the command line without any options.
sysbench 1.0.17 (using system LuaJIT 2.0.4)

Removing test files...
[root@ip-10-0-1-4 ec2-user]#
```

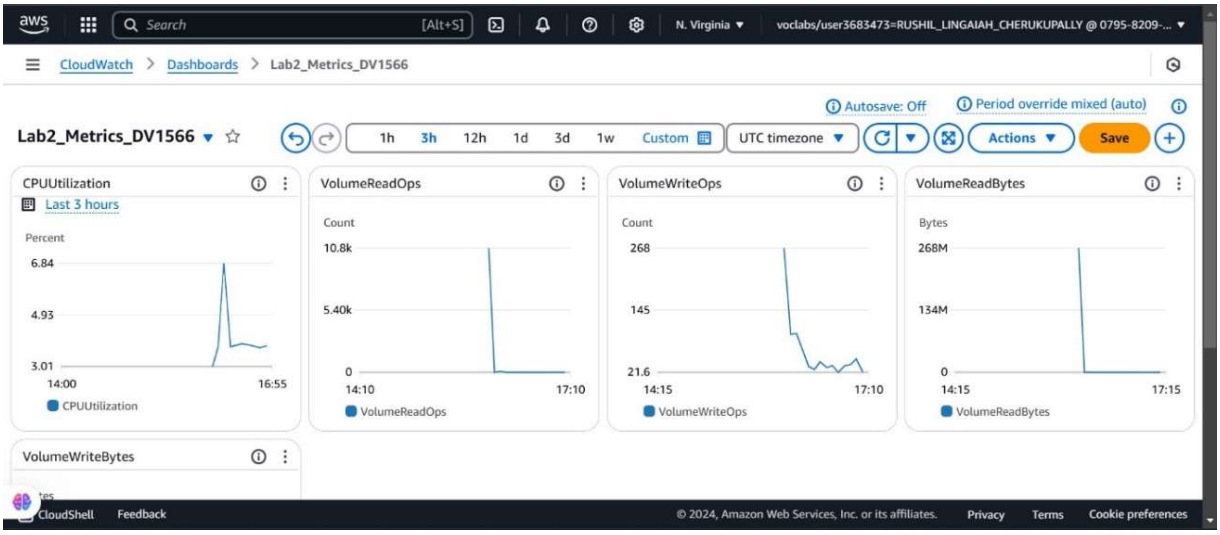
6. Monitoring with CloudWatch:

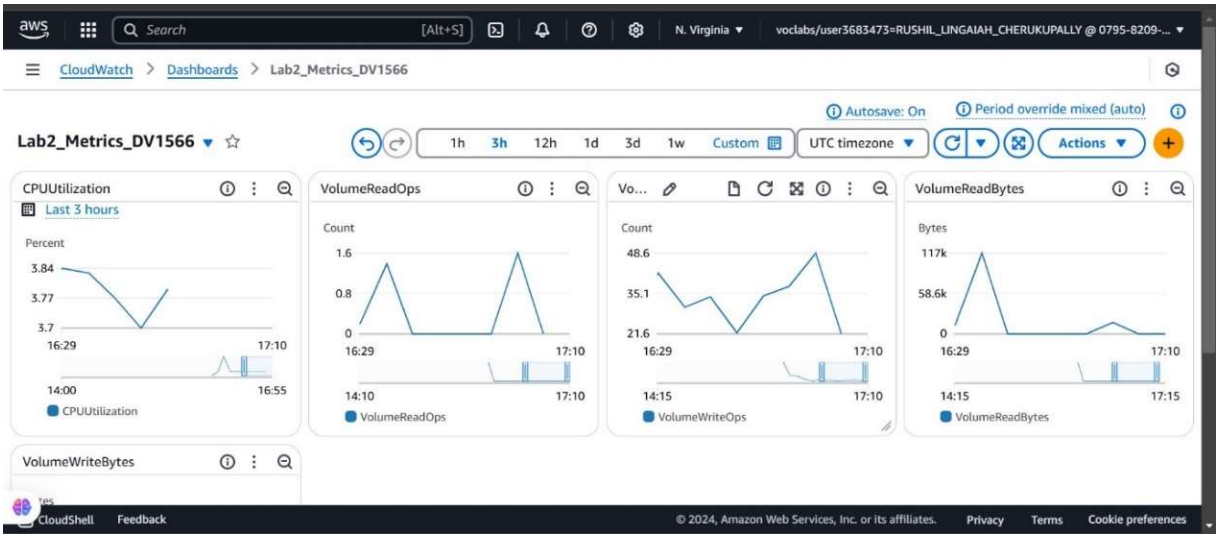
I have configured a CloudWatch dashboard named **Lab2_Metrics_DV1566** to monitor critical performance metrics for all deployed EC2 instances.

Metrics Monitored:

1. **CPU Utilization (CPU Utilization):**
    - Measures the percentage of CPU usage across all instances.
    - Observations:
        - Spikes indicate periods of high computational activity, likely due to system updates, stress tests, or workload benchmarks.
2. **Volume Read Operations (VolumeReadOps):**
    - Captures the count of read operations performed on the attached storage volumes.
    - Observations:
        - Initial high read activity indicates processes like data preparation or system checks. A subsequent decrease reflects reduced disk read intensity.
3. **Volume Write Operations (VolumeWriteOps)**:
    - Tracks the number of write operations on the storage volumes.
    - Observations:
        - Significant write activity is evident at first, likely from log files or benchmarking tests, with a gradual decline as activities completed.
4. **Volume Read Bytes (VolumeReadBytes)**:
    - Displays the total volume of data read from the disk, in bytes.
    - Observations:
        - Higher initial data reads indicate stress test preparations or file system activity, tapering off as read-intensive tasks finish.

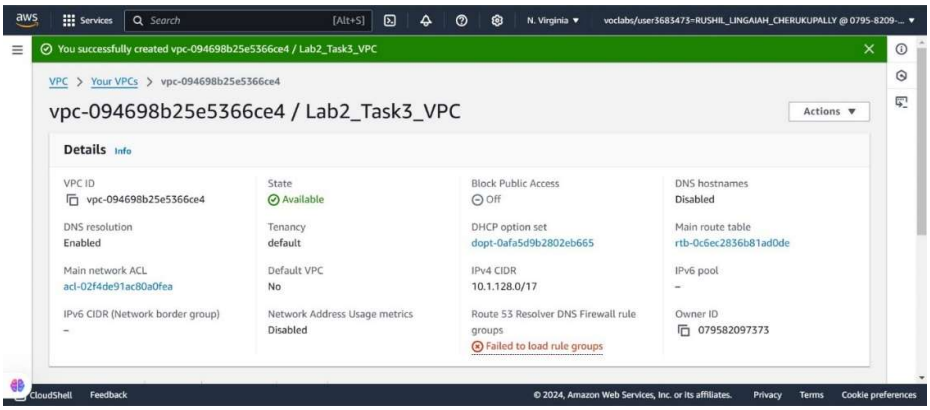
5. **Volume Write Bytes (VolumeWriteBytes)**:
    - Measures the amount of data written to the disk in bytes.
    - Observations:
        - High initial write operations correspond to system logs or stress test results being recorded, decreasing once these tasks were completed.



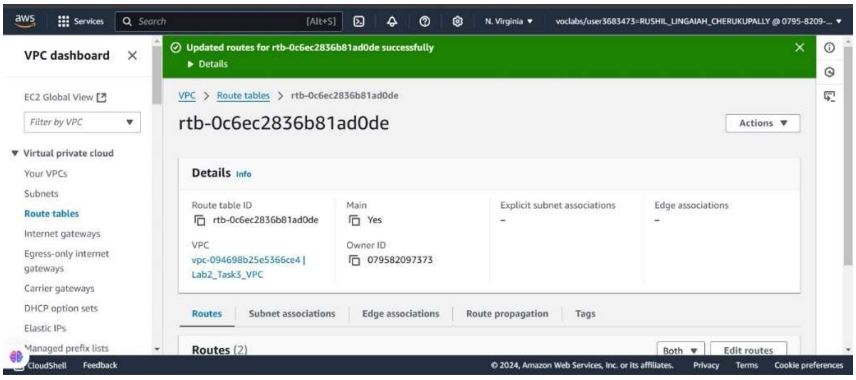


TASK 3: Enabling Communication Between Virtual Machines

1. **Setting Up a New VPC:**
   I created another VPC, **Lab2_Task3_VPC**, with a **CIDR block** of **10.1.128.0/17**, providing a secure environment for multiple EC2 instances to communicate.

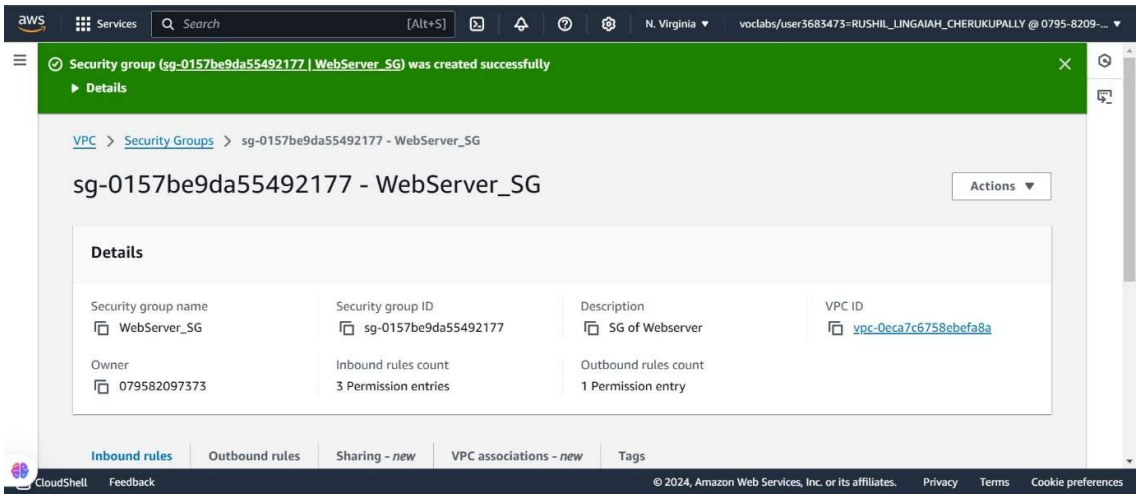


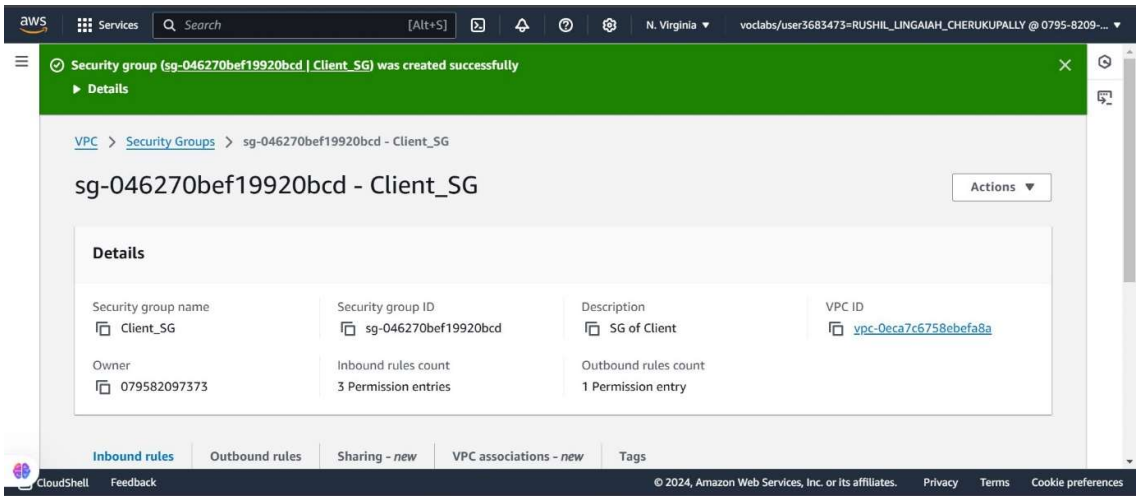
I have created the Route table for the **Lab2_Task3_VPC**, to define the rules for directing network traffic within and outside the VPC.



2. **Configuring Security Groups:**

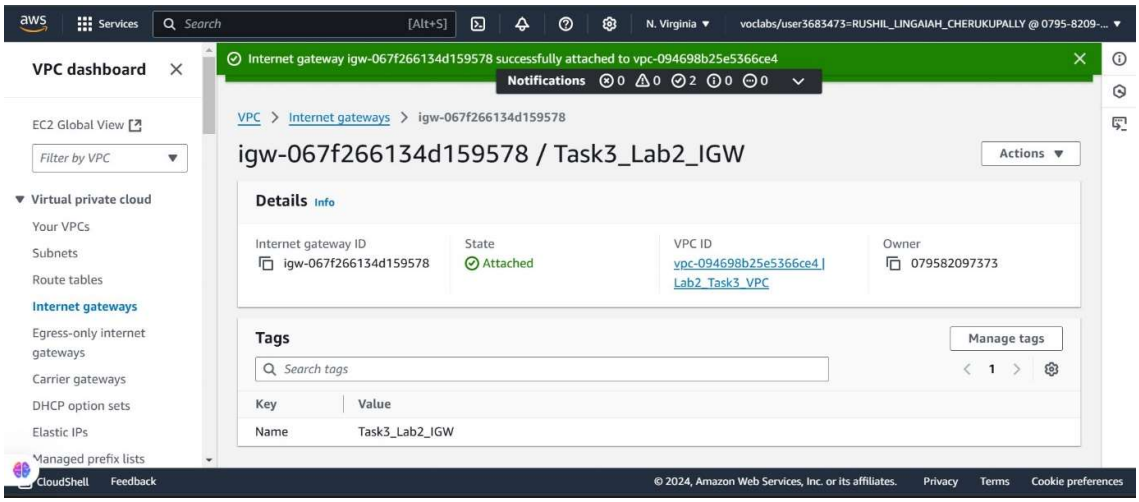
   Two security groups, **WebServer_SG** and **Client_SG**, have been created **and** set up to allow SSH, HTTP, and HTTPS access between instances while maintaining controlled traffic.





3. **Configuring Internet Gateway:**

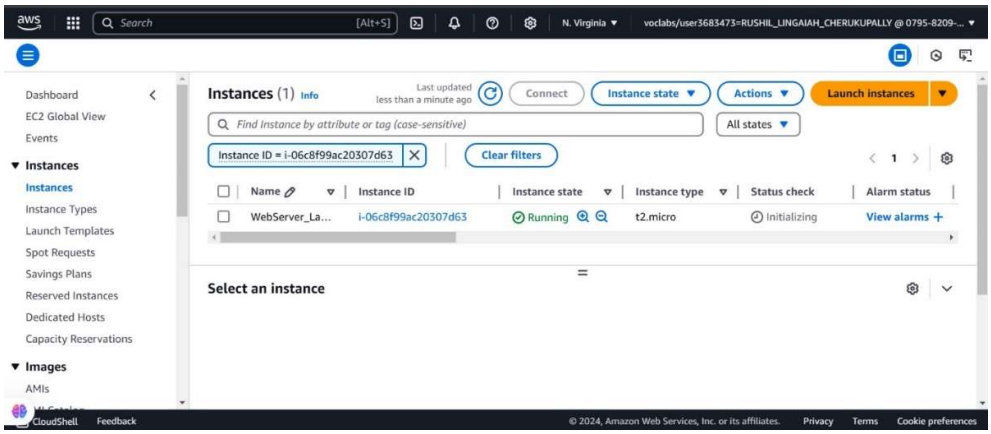
An Internet Gateway has been created with the name “Task3_Lab2_IGW” and has been attached to “**Lab2_Task3_VPC**”, for VPC to connect and communicate with the internet.



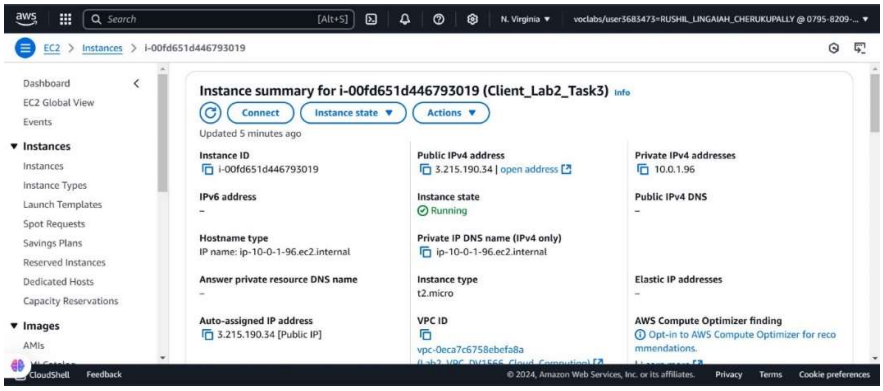
4. Launching EC2 Instances:

I launched two instances:

- **WebServer_Lab2_Task3:** Configured as a web server.



- **Client_Lab2_Task3:** Used for benchmarking the web server.



Each instance was assigned **LAB2_KEY_CLOUD** as RSA key pair for secured access.

4. Accessing the Instances via SSH

Connected to **WebServer_Lab2_Task3** (Private Subnet):

- Use an SSH client like PuTTY or your terminal to connect to **WebServer_Lab2_Task3**.
- Key and Hostname respectively:

LAB2_KEY_CLOUD.ppk and **ec2-user@ 44.192.30.66**

Connect to **Client_Lab2_Task3** (Public Subnet):

- Similarly, connect to **Client_Lab2_Task3**.
- Key and Hostname respectively:

LAB2_KEY_CLOUD.ppk and ec2-user@ 3.237.9.71

Explanation of Webserver VM IP Address

The web server VM has been assigned the public IP address 18.208.143.177, which might appear inconsistent with the VPC's CIDR block 10.0.0.0/16, a range designated for private IP addresses within the VPC. This allocation is due to AWS automatically assigning a public IP to instances in the public subnet when internet access is enabled through an attached Internet Gateway.

The public IP serves as a bridge, allowing the instance to connect to the internet and making it accessible to external users. Meanwhile, the private IP assigned from the VPC's address range is used for internal communication within the VPC. Together, these IPs ensure both secure internal networking and seamless external connectivity.

5. Configuring and Installing Required Software

On **WebServer_Lab2_Task3** (Private Subnet):

- Update the package manager:

  sudo yum update -y

- Installing Apache Web Server:

  sudo yum install httpd -y

- Start Apache:

  sudo systemctl start httpd

- Enable Apache to start on boot:

  sudo systemctl enable httpd

On **Client_Lab2_Task3** (Public Subnet):

- Update the package manager:

  sudo yum update -y

- Installing Apache utilities (ApacheBench):

sudo yum install httpd-tools -y

```
Using username "ec2-user".
Authenticating with public key "LAB2_KEY_CLOUD"
Last login: Fri Dec 13 21:46:30 2024 from 193.11.184.2
       __|  __|_  )
       _|  (     /   Amazon Linux 2
      ___|\___|___|
                      AL2 End of Life is 2025-06-30.

A newer version of Amazon Linux is available!

                      Amazon Linux 2023, GA and supported until 2028-03-15.
                        https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-1-96 ~]$ sudo su
[root@ip-10-0-1-96 ec2-user]# sudo yum update -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
No packages marked for update
[root@ip-10-0-1-96 ec2-user]# sudo yum install httpd-tools -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
Resolving Dependencies
--> Running transaction check
---> Package httpd-tools.x86_64 0:2.4.62-1.amzn2.0.2 will be installed
--> Processing Dependency: libapr-1.so.0()(64bit) for package: httpd-tools-2.4.6
2-1.amzn2.0.2.x86_64
--> Processing Dependency: libaprutil-1.so.0()(64bit) for package: httpd-tools-2
.4.62-1.amzn2.0.2.x86_64
--> Running transaction check
---> Package apr.x86_64 0:1.7.2-1.amzn2 will be installed
---> Package apr-util.x86_64 0:1.6.3-1.amzn2.0.1 will be installed
--> Processing Dependency: apr-util-bdb(x86-64) = 1.6.3-1.amzn2.0.1 for package:
 apr-util-1.6.3-1.amzn2.0.1.x86_64
--> Running transaction check
---> Package apr-util-bdb.x86_64 0:1.6.3-1.amzn2.0.1 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

================================================================================
 Package           Arch        Version                    Repository       Size
================================================================================
Installing:
 httpd-tools       x86_64      2.4.62-1.amzn2.0.2         amzn2-core       89 k
Installing for dependencies:
```

6. Perform Stress Testing

Set Up a Web Page on **WebServer_Lab2_Task3**:

- Navigate to the Apache default directory:

cd /var/www/html

- Create a simple HTML file:

echo "Hello, this is WebServer_Lab2_Task3" | sudo tee index.html

```
Transaction test succeeded
Running transaction
  Installing : apr-1.7.2-1.amzn2.x86_64                                     1/9
  Installing : apr-util-1.6.3-1.amzn2.0.1.x86_64                            2/9
  Installing : apr-util-bdb-1.6.3-1.amzn2.0.1.x86_64                        3/9
  Installing : httpd-tools-2.4.62-1.amzn2.0.2.x86_64                        4/9
  Installing : httpd-filesystem-2.4.62-1.amzn2.0.2.noarch                   5/9
  Installing : generic-logos-httpd-18.0.0-4.amzn2.noarch                    6/9
  Installing : mailcap-2.1.41-2.amzn2.noarch                                7/9
  Installing : mod_http2-1.15.19-1.amzn2.0.2.x86_64                         8/9
  Installing : httpd-2.4.62-1.amzn2.0.2.x86_64                              9/9
  Verifying  : apr-util-bdb-1.6.3-1.amzn2.0.1.x86_64                        1/9
  Verifying  : httpd-2.4.62-1.amzn2.0.2.x86_64                              2/9
  Verifying  : apr-1.7.2-1.amzn2.x86_64                                     3/9
  Verifying  : mod_http2-1.15.19-1.amzn2.0.2.x86_64                         4/9
  Verifying  : apr-util-1.6.3-1.amzn2.0.1.x86_64                            5/9
  Verifying  : mailcap-2.1.41-2.amzn2.noarch                                6/9
  Verifying  : generic-logos-httpd-18.0.0-4.amzn2.noarch                    7/9
  Verifying  : httpd-tools-2.4.62-1.amzn2.0.2.x86_64                        8/9
  Verifying  : httpd-filesystem-2.4.62-1.amzn2.0.2.noarch                   9/9

Installed:
  httpd.x86_64 0:2.4.62-1.amzn2.0.2

Dependency Installed:
  apr.x86_64 0:1.7.2-1.amzn2
  apr-util.x86_64 0:1.6.3-1.amzn2.0.1
  apr-util-bdb.x86_64 0:1.6.3-1.amzn2.0.1
  generic-logos-httpd.noarch 0:18.0.0-4.amzn2
  httpd-filesystem.noarch 0:2.4.62-1.amzn2.0.2
  httpd-tools.x86_64 0:2.4.62-1.amzn2.0.2
  mailcap.noarch 0:2.1.41-2.amzn2
  mod_http2.x86_64 0:1.15.19-1.amzn2.0.2

Complete!
[root@ip-10-0-2-28 ec2-user]# sudo systemctl start httpd
[root@ip-10-0-2-28 ec2-user]# sudo systemctl enable httpd
Created symlink from /etc/systemd/system/multi-user.target.wants/httpd.service t
o /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-2-28 ec2-user]# cd /var/www/html
[root@ip-10-0-2-28 html]# echo "Hello, this is WebServer_Lab2_Task3"| sudo tee i
ndex.html
Hello, this is WebServer_Lab2_Task3
[root@ip-10-0-2-28 html]#
```

Run ApacheBench on Client_Lab2_Task3:

- **Use ApacheBench to generate traffic:**

ab -n 2000 -c 10 http://<18.208.143.177>/index.html

- **Parameters Explained:**
  - **-n 2000: Total number of requests to send.**
  - **-c 10: Number of concurrent requests.**

```
[root@ip-10-0-1-96 ec2-user]# ab -n 2000 -c 10 http://18.208.143.177//index.html

This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 18.208.143.177 (be patient)
Completed 200 requests
Completed 400 requests
Completed 600 requests
Completed 800 requests
Completed 1000 requests
Completed 1200 requests
Completed 1400 requests
Completed 1600 requests
Completed 1800 requests
Completed 2000 requests
Finished 2000 requests


Server Software:        Apache/2.4.62
Server Hostname:        18.208.143.177
Server Port:            80

Document Path:          //index.html
Document Length:        36 bytes

Concurrency Level:      10
Time taken for tests:   2.082 seconds
Complete requests:      2000
Failed requests:        0
Total transferred:      634000 bytes
HTML transferred:       72000 bytes
Requests per second:    960.83 [#/sec] (mean)
Time per request:       10.408 [ms] (mean)
Time per request:       1.041 [ms] (mean, across all concurrent requests)
Transfer rate:          297.44 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    4   3.3      6      15
Processing:     1    6   3.6      7      17
Waiting:        1    6   3.6      7      17
```

```
=================================================================================
 Package           Arch        Version                    Repository       Size
=================================================================================
Installing:
 httpd-tools       x86_64      2.4.62-1.amzn2.0.2         amzn2-core       89 k
Installing for dependencies:
 apr               x86_64      1.7.2-1.amzn2              amzn2-core      130 k
 apr-util          x86_64      1.6.3-1.amzn2.0.1          amzn2-core      101 k
 apr-util-bdb      x86_64      1.6.3-1.amzn2.0.1          amzn2-core       22 k

Transaction Summary
=================================================================================
Install  1 Package (+3 Dependent packages)

Total download size: 341 k
Installed size: 665 k
Downloading packages:
(1/4): apr-util-1.6.3-1.amzn2.0.1.x86_64.rpm               | 101 kB   00:00
(2/4): apr-1.7.2-1.amzn2.x86_64.rpm                        | 130 kB   00:00
(3/4): apr-util-bdb-1.6.3-1.amzn2.0.1.x86_64.rpm           |  22 kB   00:00
(4/4): httpd-tools-2.4.62-1.amzn2.0.2.x86_64.rpm           |  89 kB   00:00
---------------------------------------------------------------------------------
Total                                               2.1 MB/s | 341 kB  00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : apr-1.7.2-1.amzn2.x86_64                                      1/4
  Installing : apr-util-bdb-1.6.3-1.amzn2.0.1.x86_64                         2/4
  Installing : apr-util-1.6.3-1.amzn2.0.1.x86_64                             3/4
  Installing : httpd-tools-2.4.62-1.amzn2.0.2.x86_64                         4/4
  Verifying  : httpd-tools-2.4.62-1.amzn2.0.2.x86_64                         1/4
  Verifying  : apr-1.7.2-1.amzn2.x86_64                                      2/4
  Verifying  : apr-util-1.6.3-1.amzn2.0.1.x86_64                             3/4
  Verifying  : apr-util-bdb-1.6.3-1.amzn2.0.1.x86_64                         4/4

Installed:
  httpd-tools.x86_64 0:2.4.62-1.amzn2.0.2

Dependency Installed:
  apr.x86_64 0:1.7.2-1.amzn2              apr-util.x86_64 0:1.6.3-1.amzn2.0.1
  apr-util-bdb.x86_64 0:1.6.3-1.amzn2.0.1

Complete!
```

```
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    4   3.3      6      15
Processing:     1    6   3.6      7      17
Waiting:        1    6   3.6      7      17
Total:          1   10   5.3     10      23

Percentage of the requests served within a certain time (ms)
  50%     10
  66%     14
  75%     16
  80%     16
  90%     16
  95%     16
  98%     19
  99%     22
 100%     23 (longest request)
[root@ip-10-0-1-96 ec2-user]#
```

7. Monitoring Network Traffic with CloudWatch

Navigating to CloudWatch Console and Creating a CloudWatch Dashboard:
I clicked on Create Dashboard and named it **Lab2_Task3_Cloud**. After naming it, I was prompted to add widgets to the dashboard.

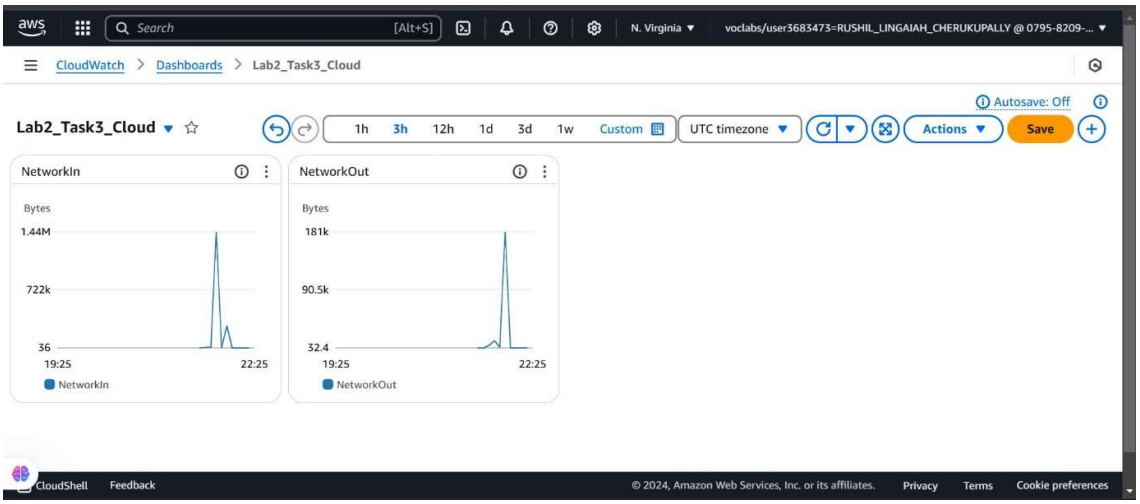
Adding Metrics for **WebServer_Lab2_Task3**:

I clicked on Add Widget and selected Line Chart as the visualization type because it's ideal for monitoring metrics over time. In the Add Metrics panel, I searched for EC2 and located the relevant metrics for my instance **WebServer_Lab2_Task3**.

Adding Relevant Metrics:
I added the following metrics to the dashboard:

- Network In: This metric tracks the incoming network traffic to the server, helping me understand how much data **WebServer_Lab2_Task3** was receiving from **Client_Lab2_Task3**.
- Network Out: This tracks the outgoing network traffic, showing how much data the server was sending in response to the ApacheBench requests.

For each metric, I selected it and clicked Add to Dashboard.



Visualizing Data:
Using the line charts on the dashboard, I monitored the following:

- Traffic spikes during the stress tests conducted using ApacheBench.
- Patterns in data transfer, including increases in traffic during the load phase and stabilization afterward.

Overall, the performance of the metrics went on as expected and completed successfully.